REPAIR, BREAKAGE, AND REFERENCE PRESERVATION IN VERIFY–REFINE WORKFLOWS

Anonymous authors
Paper under double-blind review

ABSTRACT

A verify–refine workflow does two opposing things: it repairs tasks its reference system gets wrong, and it overwrites tasks the reference already got right. Reporting only net accuracy hides both. We define *repair* and *breakage* and give the exact accounting identity relating them to the net effect, $\Delta = (1-p)r - pb$, then use it to isolate which term a design can act on. Gating a refiner on its verifier does not touch the term in which the verifier accepts a workflow's own wrong draft, because that path never reaches the refiner. It closes only when the workflow holds the reference system's actual output, in which case a regression requires a false rejection first and $b \leq \Pr(R \mid B \text{ correct})$ — a quantity obtainable without running the refinement loop, given reference outputs and their correctness labels. Across ten verifier and domain conditions under a reservation ledger that enforces per-instance caps before each call is issued, observed breakage stays under this bound while its slack ranges from nothing to a factor of four, and removing the verifier's information raises breakage from 0.140 to 0.234 at a cost of 11.8 accuracy points. The bound was identified post hoc and then stress-tested across preregistered and prospectively locked conditions. Results come from a single model deployment and a restricted class of verify–refine workflows; complete logs, the ledger and the preregistration record are released.

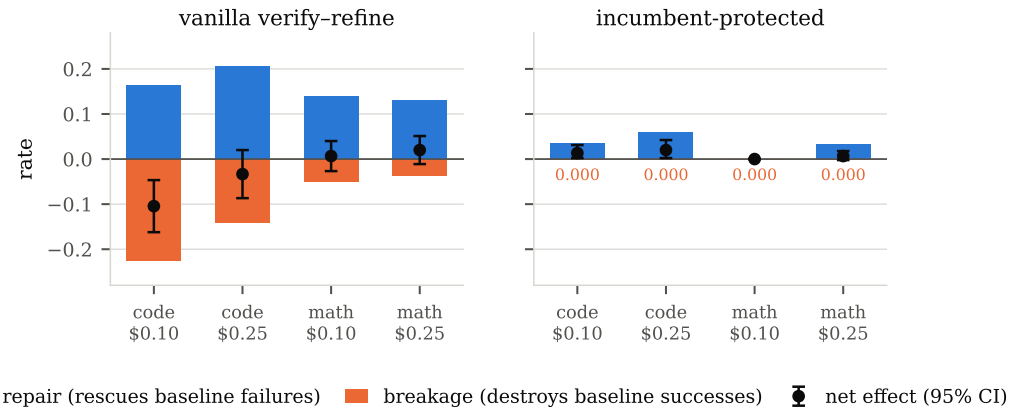

Figure 1: The same experiment, read two ways. **Left:** a standard verify–refine workflow repairs many tasks the baseline fails (blue, above the axis) and destroys many the baseline solves (orange, below), so the net effect (black, 95% CI)—the quantity the literature reports—is small and can take either sign. **Right:** protecting the incumbent answer removes the lower term entirely and the same machinery becomes a small, reliable gain. Frozen test splits, three execution seeds, matched hard budgets; panels share an axis.

1 INTRODUCTION

A large fraction of deployed language-model systems are not single calls but workflows: a model drafts an answer, a verifier inspects it, a refiner revises it, and the loop repeats until something stops

it. The pattern is old enough to have a canon—self-refinement (Madaan et al., 2023), verbal reinforcement (Shinn et al., 2023), multi-agent debate (Du et al., 2024), self-consistency (Wang et al., 2023)—and new enough that its value is still contested. Recent work reports that multi-agent structure improves the compute–accuracy frontier under matched budgets (Anonymous, 2026b); other recent work argues the opposite, that once inference compute is genuinely equalised the advantage evaporates and reported gains reflect unaccounted computation rather than architecture. Both camps run careful experiments. Both report a single number: the difference in accuracy between a workflow and its baseline.

We think that number is the problem. A workflow that adds verification and refinement does two things at once, and they point in opposite directions. It rescues tasks the baseline would have failed—the reason anyone builds such systems. It also damages tasks the baseline had already solved, because a refiner asked to improve a correct answer will sometimes take it apart. Nothing in the standard evaluation separates these. When the two are of similar magnitude, their difference is small, unstable, and sensitive to conditions that vary between papers: how much budget each call is allowed, how reliable the verifier is, how strong the first draft was. A field that reports only the difference will produce exactly the pattern we observe—careful studies that disagree.

This paper separates them. We evaluate workflows under a reservation-based budget ledger: before any node executes, an upper bound on its billable cost is reserved against the remaining per-instance caps, and a node that cannot reserve does not run. Overruns become impossible rather than merely reported, which is what makes "compute-matched" mean something. Within that protocol we measure, for every workflow and every budget, not one number but three: the repair rate on tasks the baseline fails, the breakage rate on tasks the baseline solves, and the net effect the two produce.

Figure 1 shows what this reveals. On a program-synthesis benchmark at a \$0.25 per-task budget, a three-iteration verify–refine workflow repairs 20.5% of the baseline's failures—a real and substantial capability—while breaking 14.0% of its successes. The net is -3.3 points, at $5.5\times$ the cost. Tighten the budget to \$0.10 and the same workflow becomes significantly harmful (-10.4 points): with fewer affordable iterations, repair falls and breakage rises. Withhold the verifier's signal entirely and the effect falls to -11.8 points, because a refiner with nothing to gate on rewrites correct answers along with incorrect ones. Each of these is the movement of one term in the decomposition, produced by a controlled manipulation of one variable.

The decomposition is not a model we fit; for binary outcomes it is an algebraic identity, and we are explicit about that because it determines what the identity can be used for. It cannot be validated on the data used to estimate its terms. What it does buy is exhaustiveness—every effect a workflow has on accuracy is repair or breakage, and there is no third channel—and, rearranged, a design rule: refinement pays exactly when the breakage-to-repair ratio falls below $(1-p)/p$, where p is baseline accuracy. Because the ratio is a design choice, the rule is actionable—though not in the way we first assumed. Breakage has two paths, and gating the refiner closes only one of them: a workflow that drafts its own answer can be wrong where the reference is right, and an accepting verifier passes that answer through without refinement ever running. The path closes only when the workflow's incumbent *is* the reference system's output. Under that condition, which we verify holds exactly in our runs, breakage is 0.000 in all six in-domain cells and is bounded by the verifier's false-rejection rate.

A design principle derived by us from our own decomposition and then tested by us is a route by which an intuition can be mistaken for a finding. We therefore built a budget-safe automated search over a restricted workflow language—not to propose a new optimiser, but to ask whether a procedure with no access to our reasoning arrives at the same place. In the code family it does: every search arm converges on a draft behind a verifier gate, and a single policy conditioned on remaining budget matches per-budget static search, at both searched budgets and at an intermediate budget it never saw, using half the search cost. The searcher is ours, so this bounds our design bias rather than eliminating it. In the math family it does not, and we report that too.

We take the reliability of these claims as seriously as the claims themselves. Every confirmatory hypothesis was written, committed and git-tagged before the corresponding frozen split was executed (Nosek et al., 2018), and we report all of them as they landed: four Part-I claims supported and one refuted, three Part-II claims not supported as stated, and a locked out-of-domain prediction that failed—because, as §7 diagnoses, our own out-of-domain split had been built without visible tests,

so it varied domain and verifier availability at once. An automated audit recomputes every numeric claim in this paper from raw per-task logs.

Three contributions follow, stated at their real size. First, the finding a gate alone does not deliver: *whose answer* a verifier protects, not merely whether one gates a refiner, determines whether a regression is possible at all. For workflows that retain the reference system's actual output and modify it only after verifier rejection, breakage is bounded above by the verifier's false-rejection rate on reference-correct outputs — an inclusion argument rather than a theorem with independent force, but one that turns a design question into a measurable, pre-deployment check. Second, the accounting language that makes this precise: repair and breakage are an exhaustive decomposition of how a workflow changes binary task accuracy, and reporting both costs nothing extra for anyone already running such a comparison. Third, an *empirical characterisation*: we measure repair, breakage and their verifier dependence under reservation-enforced per-instance budgets across code, mathematics and logic tasks. This paper does not resolve which inference strategy is best at matched cost; it resolves an orthogonal, necessary question — how a candidate workflow changes baseline-correct and baseline-wrong instances, and whether the regression risk can be bounded before deployment. An equal-cost comparison decides whether to adopt a workflow; the diagnostic and certificate here decide how to evaluate and safely adopt one. §6 reports a post-hoc cost-and-accuracy comparison against a sampling baseline built at no new cost, though not one we call matched.

2 Background and related work

Iterative refinement and its discontents. Self-Refine (Madaan et al., 2023) and Reflexion (Shinn et al., 2023) established the generate–critique–revise loop as a default building block, and multi-agent debate (Du et al., 2024) and self-consistency (Wang et al., 2023) extended the idea to populations of samples. Sceptical results arrived quickly: Huang et al. (2024) showed that models cannot reliably self-correct reasoning without external feedback, and attributed apparent gains to oracle labels leaking into the correction signal. Our decomposition puts a quantity behind that critique. Self-correction without external signal is exactly the regime in which b is unconstrained, and we measure what it costs: -11.8 points when the verifier is stripped of information, with breakage rising from 0.140 to 0.234.

Automated workflow design. A body of work searches over agent structures rather than fixing them by hand: GPTSwarm (Zhuge et al., 2024) optimises agent graphs jointly over prompts and edges; ADAS (Hu et al., 2024) has a meta-agent write agent code; AFlow (Zhang et al., 2025b) searches code-level workflows by tree search; AgentSquare (Shang et al., 2024) searches a modular design space; EvoFlow (Zhang et al., 2025a) evolves heterogeneous workflows under cost and accuracy objectives; EvoAgentX (Wang et al., 2025) packages several such optimisers. These optimise *within* a design space; our question is prior to it, what determines whether any structure in such a space is worth its cost.

Budget-aware inference. Li et al. (2026) condition inference-tree node selection on the remaining-resource ratio, and Anonymous (2026a) make workflow *search* cost-aware through prefix reuse and caching. The budget conditioning we search over acts on workflow control flow rather than on tree node selection, and our ledger enforces caps before execution rather than accounting for them afterwards—a distinction that matters, because a workflow prevented from overspending behaves differently from one that overspends and is billed for it.

Test-time compute allocation. For a single model the tradeoff between sampling more and refining longer is well studied: Snell et al. (2024) show the optimal allocation depends on problem difficulty and Anonymous (2025) on verification granularity. This is the closest antecedent to our budget floor. What it does not do, and what we add, is separate the two directions of a workflow's effect. The crossovers reported there are crossovers in a net quantity, and the mechanism we identify—that structure destroys correct answers at a rate governed by verifier specificity—is invisible to it.

Abstention, deferral and conservative updates. Declining to act on a low-confidence prediction is old: the reject option dates to Chow (1970) and reappears as selective classification (Geifman & El-Yaniv, 2017) and learning to defer (Madras et al., 2018; Mozannar & Sontag, 2020). Detector

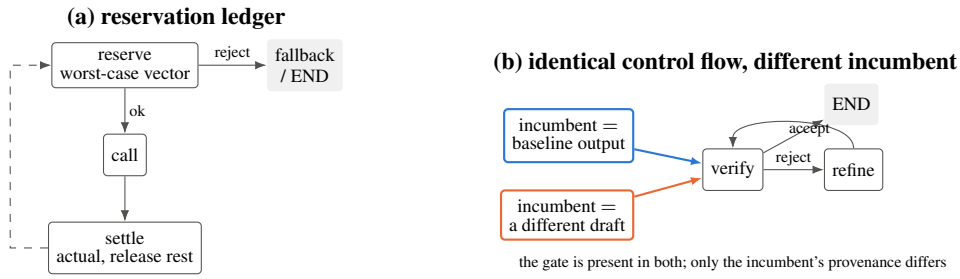

Figure 2: **(a)** Every metered call reserves an upper bound on its billable cost before executing; a node that cannot reserve does not run, and control follows a fallback edge. Overruns are prevented rather than recorded. **(b)** The two workflows we compare have *identical* control flow: both terminate on an accepted draft and refine only on rejection. What differs is where the incumbent comes from. One reuses the reference baseline's own output; the other independently produces a different draft (a different prompt at a different temperature). §4 shows this is the operative distinction, and §8 records that our two arms also differ in prompt and temperature, so the isolated contribution of each is not established.

cascades (Viola & Jones, 2001) and their language-model descendants (Chen et al., 2023) gate expensive stages on a cheap one; conservative policy updates (Thomas et al., 2015; Garcelon et al., 2020) refuse changes that cannot be shown not to regress; and regression testing (Leung & White, 1989) exists to stop a change from breaking what already worked. Reference preservation as we state it is a member of this family: a guarded fallback whose guard is a verifier and whose fallback is a reference system's own output. What we add is not the idea of guarding but its consequences under explicit inference budgets — an exhaustive accounting of where a workflow's effect comes from, a bound tying the residual to a verifier property that can be measured beforehand, and the observation that the guard is useless unless the guarded object is the reference's actual output. Calibration of model self-assessment (Kadavath et al., 2022) is the missing ingredient our verifier taxonomy stands in for.

Evaluation. We use MBPP+ and HumanEval+ (Austin et al., 2021; Chen et al., 2021) in the rigorous-test formulation of EvalPlus (Liu et al., 2023), MATH (Hendrycks et al., 2021) and BIG-Bench Hard (Suzgun et al., 2023); multi-fidelity screening follows successive halving (Jamieson & Talwalkar, 2016; Li et al., 2018), and intervals are stratified paired bootstrap (Efron & Tibshirani, 1994) with Holm correction (Holm, 1979).

3 Hard budgets and a reservation ledger

A deployment budget is a vector of caps on input tokens, output tokens, LLM calls, tool calls, wall-clock seconds and dollars. We use five profiles from \$0.02 to \$0.25 per task. Two are searched—`tight` (\$0.10, four calls) and `loose` (\$0.25, eight calls)—and `unseen` (\$0.15) is touched only at final evaluation, enforced by an assertion in the search path that fails if any selection code reads it.

Before a node executes, an upper bound on its billable vector is computed and *reserved* against the remaining caps. If the reservation fails the node does not run: control follows a fallback edge or terminates with the current answer. After the call, actual usage is settled and the surplus released, and a settlement exceeding its own reservation is surfaced as a defect rather than absorbed. Concurrent calls, such as the k branches of a vote, reserve their full aggregate before any of them starts. Figure 2(a) shows the discipline and Appendix B proves the resulting invariant and reports a 3,000-workflow fuzz sweep with zero cap violations in any dimension.

This is not only hygiene: under post-hoc accounting a workflow that overspends is recorded as expensive, whereas under reservation it is prevented, and the prevention becomes an observable—reserve rejections turn out to be the visible face of the budget floor. Compute matching is enforced rather than audited.

Model. All experiments use GPT-4o. Unless a node specifies otherwise, generation and refinement use temperature 0.7; the protected first draft uses temperature 0. No model is fine-tuned anywhere.

Verifiers. We distinguish two regimes and never conflate them. In the code family the workflow's verifier runs the task's own visible tests: environment feedback that is part of the task specification, with hidden extended tests reserved for the external grader. In the math and logic families no oracle is available to the workflow, and the verifier is a gold-free self-consistency check that re-derives the answer independently and compares. Gold answers never enter workflow state in any family.

Tasks. Code: MBPP+, split 120/40/150 for search, selection and confirmation, with HumanEval+ held out. Math: MATH levels 4–5 over four subjects, split 120/40/150, with two further subjects held out. Logic: BIG-Bench Hard, 120 tasks, reserved for §7 and used nowhere else. Splits were generated once with a fixed seed, hashed, and never regenerated. All final numbers use three execution seeds, seed-averaged per task, with stratified paired bootstrap intervals over $10{,}000$ resamples.

4 THE REPAIR–BREAKAGE DECOMPOSITION

Let B be a baseline and W a workflow evaluated on the same tasks under the same budget. Write $p = \Pr[B \text{ correct}]$ and

$$r = \Pr[W \text{ correct} \mid B \text{ incorrect}], \qquad b = \Pr[W \text{ incorrect} \mid B \text{ correct}], \tag{1}$$

so that the net change in accuracy is

$$\Delta \;=\; \Pr[W] - \Pr[B] \;=\; (1-p)\,r \;-\; p\,b. \tag{2}$$

Equation 2 holds exactly, by the law of total probability, for binary per-task outcomes. We state this plainly because it determines what the equation is good for. It cannot be "validated" by estimating r and b on a dataset and checking that it reproduces Δ there; that check is tautological, and we do not present one. What it establishes is that the decomposition is *exhaustive*: every effect a workflow has on accuracy is repair or breakage. A literature that reports only Δ is reporting a difference of two quantities each of which can be an order of magnitude larger than the difference itself.

Rearranged, equation 2 gives a condition for when refinement is worth running:

$$\frac{b}{r} \;<\; \frac{1-p}{p}. \tag{3}$$

Two regimes follow. With an oracle verifier that never rejects a correct answer, $b \to 0$ and refinement cannot hurt. With a verifier carrying no information every answer is flagged, b rises to the unconditional rate at which revision damages correct answers, and equation 3 fails whenever the baseline is strong—which, for a competent model on a standard benchmark, it is. The threshold is punishing: $p = 0.73$ demands $b/r < 0.38$ and $p = 0.92$ demands $b/r < 0.09$.

The rule is actionable because its left-hand side is a design choice, though not the one we first supposed. Consider a workflow that emits an incumbent I, accepts it when a verifier does, and otherwise refines it into J. Writing R for rejection, breakage decomposes into two paths:

$$b = \underbrace{\Pr(\neg R,\ I \text{ wrong} \mid B \text{ correct})}_{\text{a wrong incumbent the verifier accepted}} + \underbrace{\Pr(R,\ J \text{ wrong} \mid B \text{ correct})}_{\text{refinement failed after a rejection}}. \tag{4}$$

The first term is the one that matters and the one the gate does not touch: if the workflow's incumbent is some answer of its own, it can be wrong on a task the reference solves, and an accepting verifier then hands that wrong answer straight through. No refinement is involved. Gating the refiner cannot remove this path, because the path never reaches the refiner.

It vanishes under one condition. If $I = B$ — the workflow's incumbent is the reference system's actual output, not merely an answer produced under the same configuration — then B correct implies I correct, the first term is identically zero, and

$$b = \Pr(R,\ J \text{ wrong} \mid B \text{ correct}) \;\le\; \Pr(R \mid B \text{ correct}). \tag{5}$$

We call this *reference preservation*. It is a condition on provenance rather than a control-flow feature: a workflow does not regress against a reference unless the verifier first falsely rejects that reference's own answer. Equality in equation 5 holds only if every false rejection ends in a wrong answer.

The distinction matters because $I = B$ is not implied by configuring the incumbent's generator identically to the baseline. Nominally deterministic decoding does not guarantee byte-identical outputs across calls. In our runs the condition does hold exactly — among executions where the verifier accepted, the workflow's output is byte-identical to the baseline's in $402/402$ code and $396/396$ math cases — but it holds because both calls resolve to the same response-cache entry, not because determinism was established. A deployment that wants the guarantee should reuse the reference's stored output rather than regenerate it.

We are explicit about what equation 5 is and is not. Given the premise that a different answer is reachable only after a rejection, breakage events are a subset of false-rejection events, so the inequality follows by inclusion rather than by any deeper argument. Checking it against data tests whether an implementation satisfies the premise, and how much slack the bound leaves — not whether a contingent law holds. The premise has four parts, and a deployment wanting the guarantee must satisfy all of them: the incumbent is the reference system's *actual output*, not a fresh call under the same configuration; an accepted incumbent is returned unchanged; every path to a different answer passes through a rejection; and the bound is stated on reference-correct instances, so estimating it needs correctness labels or a grader, not the verifier alone. Table 1 reports it across all ten conditions we ran, spanning verifiers that almost never reject a correct draft and verifiers that reject every draft. It is respected everywhere, which is what a correct implementation should produce, and the informative part is not that it holds but how much slack it leaves: nearly none under moderate verifier noise, and a factor of four when the verifier is uninformative, because refinement sometimes recovers an answer it has just discarded. The practical reading is narrower than "measure your verifier": estimating $\Pr(R \mid B \text{ correct})$ needs the reference system's own outputs, labels or a grader saying which of them are correct, and data representative of deployment. What it does not need is a run of the refinement loop, so the cap can be obtained before committing to one. That gives a decision procedure rather than a description: if a deployment can tolerate a breakage risk of at most ϵ, then before enabling refinement, estimate a high-confidence upper bound on the verifier's false-rejection rate over reference-correct outputs and require that bound—not the point estimate—to fall below ϵ. Table 1 carries out exactly that estimate, and Figure 6 plots the underlying inequality. The gap between the two columns is the point: at code/`loose` the false-rejection rate is 0.009, but 107 labelled tasks carrying one rejection certify only 0.044, and at math OOD 27 tasks certify nothing below 0.263. A deployment with $\epsilon = 0.02$ is licensed by no condition we ran, however small its measured rate. That is a statement about our sample sizes rather than about the verifiers, and it is the honest cost of using the bound as a gate: certifying a low breakage risk takes enough labelled reference-correct data to rule one out, which is more than reporting a rate takes. Sections 5 and 7 test whether this works, where it fails, and whether the parameters transfer. They do not transfer for unconstrained workflows; they do for constrained ones, which is a claim about our construction rather than a law.

5 CONFIRMATORY EXPERIMENTS

Five claims were written, committed and tagged (`prereg-freeze-partI`) before any frozen test split was executed. We report all five.

The net effect is zero and the terms are not (C1, supported). At `loose`, vanilla verify–refine differs from its baseline by $-0.033\,[-0.087, +0.020]$ in code and $+0.020\,[-0.011, +0.051]$ in math, at $5.5\times$ and $2.2\times$ the cost. Neither interval excludes zero: the workflow buys nothing measurable at several times the price. The decomposition shows what is happening underneath—in code it repairs a fifth of the baseline's failures and destroys a seventh of its successes.

Structure has an entry fee (C2, supported). At `tight`, code vanilla is -0.104 $[-0.162, -0.047]$. Moving `loose`→`tight`, repair falls $0.205 \rightarrow 0.162$ while breakage rises $0.140 \rightarrow 0.224$: fewer affordable iterations make the loop likelier to leave a task mid-revision, having already discarded a working answer (Figure 4, Appendix D). The ledger shows why: 35 of 150 tasks could not fund a planned refinement call, refused at three or four of four calls used, dollars

Table 1: The planning quantity, per condition. FRR is the verifier's false-rejection rate on baseline-correct tasks; its 95% UCB is the one-sided Clopper–Pearson limit, computed on the task-level indicator "rejects in at least one seed" so that it is conservative with respect to within-task correlation and stays valid when no event is observed. Breakage is the observed rate for the same workflow, and slack is FRR minus breakage — how much equation 5 over-predicts. A deployment tolerating breakage ϵ enables refinement only where the UCB column, not the FRR column, falls below ϵ.

Condition	n correct	FRR	FRR 95% UCB	breakage	slack
math, self-check, tight	100	0.000	0.030	0.000	0.000
code, oracle tests, loose	107	0.009	0.044	0.000	0.009
code, oracle tests, tight	107	0.009	0.044	0.000	0.009
math, self-check, loose	100	0.013	0.062	0.000	0.013
BBH, self-check, loose	104	0.019	0.111	0.006	0.013
code OOD, tests restored	62	0.048	0.120	0.000	0.048
math OOD, self-check	27	0.049	0.263	0.025	0.025
code, 50% tests, loose	107	0.156	0.370	0.146	0.009
code, no tests, loose	107	1.000	1.000	0.234	0.766
code OOD, no tests	88	1.000	1.000	0.114	0.886

Table 2: Frozen test splits, 150 tasks per family, three execution seeds. Baselines are the strongest single-call structure per family (direct prompting for code, chain-of-thought for math). *Vanilla* is generate→verify→refine with up to three iterations; *protected* is the same loop with a temperature-zero incumbent replaced only on verifier rejection. Intervals are stratified paired bootstrap 95% CIs. The `unseen` ($0.15) budget behaves between the two shown and is reported in Appendix D.

Family	Budget	Workflow	Δ (95% CI)	repair	breakage	$/task
code	tight	vanilla	-0.104 $[-0.162, -0.047]$	0.162	0.224	0.0062
code	tight	protected	$+0.013$ $[+0.002, +0.031]$	0.034	**0.000**	0.0022
code	loose	vanilla	-0.033 $[-0.087, +0.020]$	0.205	0.140	0.0090
code	loose	protected	$+0.020$ $[+0.002, +0.042]$	0.060	**0.000**	0.0025
math	tight	vanilla	$+0.007$ $[-0.027, +0.040]$	0.140	0.050	0.0076
math	tight	protected	$+0.000$ $[+0.000, +0.000]$	0.000	**0.000**	0.0075
math	loose	vanilla	$+0.020$ $[-0.011, +0.051]$	0.129	0.037	0.0153
math	loose	protected	$+0.007$ $[-0.002, +0.018]$	0.032	**0.000**	0.0153

at $0.008 against a $0.10 cap — the binding constraints are calls and output tokens, not dollars. Since our profiles move several caps together, this is a crossover across composite profiles, not a named resource's entry fee; a single-dimension scan is required and we do not have one.

Breakage is controlled by the verifier (C3, supported). Masking visible tests at $1.0 \to 0.5 \to 0.0$ moves the code/`loose` effect monotonically: -0.033, -0.033, -0.118 $[-0.178, -0.058]$, breakage climbing $0.140 \to 0.234$. Masking changes only what the verifier sees —a controlled manipulation of the verifier, not the model, budget, or task set (Figure 3, Appendix D) — but does not isolate false-rejection as *the* operative quantity, since removing tests also changes false acceptance and refinement frequency: verifier information is causally upstream of both terms, not mediated by one summary statistic. It also gives the sceptical literature a mechanism: refinement without external signal does not merely fail to help, it destroys correct work.

C4, unsupported as preregistered. We registered the claim that adding incumbent protection—understood as a structural change, gating the refiner on the verifier—would eliminate breakage. An audit of our own implementation, carried out after the confirmatory runs, showed that both compared workflows already had that gate: their control flow is identical edge for edge. The comparison therefore never manipulated the variable the claim was about, and we mark C4 unsupported as stated rather than restate it to fit what the comparison did manipulate.

What the comparison did manipulate, and a post-hoc proposition. The arms differed in where the incumbent came from (Figure 2(b)): in the reference-preserving arm the first call, configured

Table 3: Reference preservation against the three-sample selection built from the same drafts, code and math loose. $/task is each arm's own logged cost.

	baseline p	accuracy	$/task
code, visible-test selection	0.727	0.733	0.00489
code, reference-preserving	0.727	0.747	0.00251
math, self-consistency vote	0.733	0.753	0.02057
math, reference-preserving	0.733	0.740	0.01532

exactly as the baseline's, resolves to the same cache entry and returns byte-identical text ($402/402$ code and $396/396$ math outputs match, among verifier-accepted executions). Under that condition equation 4 gives a proposition we state as *post hoc*: if a workflow emits the reference's own output unchanged on acceptance and reaches a different answer only after rejection, acceptance-path regression is impossible and $b \leq \Pr(R \mid B \text{ correct})$. Consistent with it, no breakage occurs in any reference-preserving cell; at code/`loose` a 95% Clopper–Pearson upper bound over 107 tasks is 0.028 against a measured rate of 0.009 (zero events do not certify below the rule of three). Two cautions: $I = B$ held via caching, not by design, and the guarantee depends on $I = B$ itself, so production systems should assign the stored output directly; and a proposition formed after seeing the data cannot be confirmed by the runs that motivated it — which is why we then ran it as a controlled experiment, below.

The three-arm causal test. Three arms, identical downstream structure, differing *only* in draft provenance, cross-arm caching off: *explicit reuse* of the reference's output (an `assign` node, no call); *same-policy regeneration*, a fresh call at the reference's exact prompt and temperature; *different-policy regeneration*, the vanilla workflow's own first-draft policy. Breakage is now observed, not estimated. Code is monotone in provenance distance: 0.000, 0.028, 0.109. Math is not: 0.003, 0.027, 0.017 — same-policy noise costs *more* than a different policy. Math's explicit-reuse figure is not exactly zero either, consistent with equation 5: its verifier is a non-oracle self-check with a nonzero false-rejection rate, so even $I = B$ can be routed to refine and come back wrong. We do not read this as "more different costs more, monotonically" — that is code-specific — only as what §4 claims: explicit reuse closes the accepting path in both domains, by a wide margin over either regeneration. Appendix L has the full breakdown, including the two cheaper reanalyses this test was designed to go beyond: a same-policy leak recovered at zero cost from stored seeds, and a different-policy exposure estimated from one generation call per task without running verify or refine, whose estimate (0.110) lands almost exactly on this test's directly observed figure (0.109).

Repair is not bounded by verifier type (C5, refuted). We predicted oracle-verified repair would strictly exceed non-oracle repair, math's interval containing zero. Observed: code 0.060 $[0.000, 0.137]$, math 0.032 $[0.000, 0.075]$ — overlapping substantially. Gold-free self-checking produces real repair; we claim no more.

A restricted search over our own design space. A search space specified in advance never tests reuse of a stored reference output, so we do not read its outcome — three of five preregistered claims fail — as mechanism evidence; full results in Appendix G.

6 Post-hoc comparison with three-sample alternatives

Not preregistered: added after review, using data already on disk rather than new inference. Each frozen-test task was already drafted independently three times to compute the seed-averaged baseline — a $k = 3$ pool at no additional cost. We build one selection per task by a rule fixed in advance and specific to each domain, not one method under two names: for math, *self-consistency majority voting* over the three normalized answers, ties broken by seed index; for code, *visible-test-filtered selection* — the lowest-indexed draft passing the visible feedback tests (fallback seed 0), scored against the held-out grading tests. Neither is matched cost: three calls run 1.3–2× what the protected arm's own logs show it spent, since its cheapest path costs nothing.

Point estimates favor reference preservation in code and three-sample selection in math, but a task-paired bootstrap on the difference puts zero inside both intervals ($+0.013$ $[-0.007, +0.036]$ code, -0.013 $[-0.036, +0.007]$ math) — neither distinguishable from noise here. In code, reference preservation's point estimate is higher *and* cheaper; in math, three-sample is higher at 34% higher cost; a study powered to resolve this, at independently drawn pools, remains open (§8).

7 External validity

Parameters do not transfer; the mechanism does. Before computing any out-of-domain effect we committed (tag `ood-predictions-locked`) predictions for HumanEval+ and held-out math subjects, transferring r and b from the in-domain frozen split. They failed — mean absolute error 0.061, sign agreement $6/8$ — and the failure was our construction: the out-of-domain code split had empty visible tests, so its verifier ran in the no-signal regime while the transferred parameters came from the oracle regime, varying domain *and* verifier availability at once. Matched no-signal parameters do not rescue it either (errors 0.09–0.15), so we do not make the quantitative transfer claim. Restoring visible tests from HumanEval+ docstring examples ($68/100$ tasks) isolates domain: breakage returns to 0.000 (Table 4), and prediction is accurate for protected workflows (errors 0.002, 0.000) while remaining badly wrong for vanilla ones (0.169, 0.108) — the constrained-versus-unconstrained asymmetry of §4.

A prospective test in a third domain. Everything above is retrospective. To test the account prospectively we chose a domain no part of this project had touched—BIG-Bench Hard logical deduction, shuffled-object tracking and date understanding (Suzgun et al., 2023), 120 multiple-choice tasks with exact-match grading and a gold-free verifier—committed our predictions (`prospective-locked`), and only then ran anything. They were structural, since the parameters had already been shown not to transfer: breakage ≤ 0.02 under reference preservation, materially higher without it, and a non-negative effect. The first and third hold in all four cells (0.000, 0.006, 0.000, 0.003), in a demanding regime where the baseline is right 92–94% of the time. The second does not: at `loose` the unprotected variant also breaks only 0.006, because breakage requires that refinement be *able* to destroy a correct answer and single-label answers leave little to destroy. At `tight`, where refinement acts under pressure, the gap reappears (0.026 against 0.000). Reference preservation is insurance whose premium is visible only where the risk is.

8 Limitations

We group these by what they threaten, because they are not equally serious and a flat list would hide which ones matter.

Bearing on the central mechanism

One sampling baseline, not matched, not exhaustive. §6's post-hoc, same-pool comparison is not matched cost and not distinguishable from noise at this sample size. Verifier reranking and retry-with-fallback are untested; our own ledger equalises the budget *cap* rather than realized spend, and the unprotected workflow spends $5.5\times$ the baseline. A genuinely matched study, with independently drawn pools rather than one pool of three dependent seeds, stays open.

One model; scope and measurement. A single deployment is thin against the disagreement we open by addressing; p, r, and plausibly b depend on model quality, and transfer tests show them unstable across domains. A second family, Kimi K3 (Appendix M), replicates the qualitative verifier-sensitivity pattern across all ten conditions, not just its magnitudes. The verifier taxonomy, budget floor, and two defects are quantified in Appendices I–J.

9 Conclusion

Whether agent workflow structure helps has no budget-free answer, and reporting a net effect conceals why: structure repairs and structure breaks, and under matched caps the two are comparable,

so their difference is small and reads differently depending on where one measures. Separated, equation 4 says the design question is not whether to gate the refiner — gating leaves untouched the path where a verifier accepts a workflow's own wrong draft — but whose answer the workflow is holding when the verifier speaks. What we establish is narrower than the framing that motivates it and, we think, more usable: an exhaustive accounting of how a workflow moves accuracy, a condition under which one of its two paths provably closes, a measurable quantity that caps the other, and a characterisation of how both respond to budget and verifier signal under caps that are enforced rather than reported. A post-hoc, same-pool sampling comparison (§6) puts zero inside both arms' paired accuracy-difference interval, distinguishing neither at this sample size; a genuinely matched, adequately powered study, at independently drawn pools, remains open, and we have released the ledger, the logs and the preregistration record so it can be run against our numbers.

Reproducibility statement

All splits are hashed and frozen. Every confirmatory claim was git-tagged before the corresponding split was executed (`prereg-freeze-partI`, `prereg-freeze-partII`, `ood-predictions-locked`, `prospective-locked`). The anonymised repository contains the workflow language and its validator, the reservation ledger with its fuzz harness, every search registry with parent and mutation lineage, all raw per-task traces under both gradings, the preregistration with its append-only deviation log including the refuted claims, and a single command that recomputes every table and figure in this paper from raw logs and exits non-zero on any disagreement.

AI use statement

Language models were used as coding assistants during implementation, and are the subject of study. All design decisions, preregistered hypotheses, analyses and claims are the authors' own, and every reported number is recomputed from raw logs by the repository's audit script.

References

Anonymous. Rethinking optimal verification granularity for compute-efficient test-time scaling. *arXiv preprint arXiv:2505.11730*, 2025.

Anonymous. Agent-UCT: Upper confidence bounds applied to trees for agentic workflow optimization with cost-awareness. *arXiv preprint arXiv:2607.24162*, 2026a.

Anonymous. Multi-agent reasoning improves compute efficiency: Pareto-optimal test-time scaling. *arXiv preprint arXiv:2605.01566*, 2026b.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

C Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*, 2024.

Bradley Efron and Robert J Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1994.

Evrard Garcelon, Mohammad Ghavamzadeh, Alessandro Lazaric, and Matteo Pirotta. Conservative exploration in reinforcement learning. In *Artificial Intelligence and Statistics*, 2020.

Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.

Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations*, 2024.

Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial Intelligence and Statistics*, 2016.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Hareton K N Leung and Lee White. Insights into regression testing. In *IEEE Conference on Software Maintenance*, 1989.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018.

Y. Li et al. Spend less, reason better: Budget-aware value tree search for LLM agents. *arXiv preprint arXiv:2603.12634*, 2026.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023.

David Madras, Toniann Pitassi, and Richard Zemel. Predict responsibly: Improving fairness and accuracy by learning to defer. In *Advances in Neural Information Processing Systems*, 2018.

Hussein Mozannar and David Sontag. Consistent estimators for learning to defer to an expert. In *International Conference on Machine Learning*, 2020.

Brian A Nosek, Charles R Ebersole, Alexander C DeHaven, and David T Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018.

Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. AgentSquare: Automatic LLM agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics*, 2023.

Philip S Thomas, Georgios Theocharous, and Mohammad Ghavamzadeh. High-confidence off-policy evaluation. In *AAAI Conference on Artificial Intelligence*, 2015.

Paul Viola and Michael Jones. Rapid object detection using a boosted cascade of simple features. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2001.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Yingxu Wang, Siwei Liu, Jinyuan Fang, and Zaiqiao Meng. EvoAgentX: An automated framework for evolving agentic workflows. In *Proceedings of EMNLP: System Demonstrations*, 2025.

Guibin Zhang, Kaijie Niu, Yusen Ma, Zhixun Yang, Jiaxuan Cheng, and Xiaojiang Wang. EvoFlow: Evolving diverse agentic workflows on the fly. *arXiv preprint arXiv:2502.07373*, 2025a.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. In *International Conference on Learning Representations*, 2025b.

Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. GPTSwarm: Language agents as optimizable graphs. In *International Conference on Machine Learning*, 2024.

A WORKFLOW LANGUAGE

A workflow is a JSON graph validated statically before any spend. Node whitelist: `generate`, `refine`, `vote` ($k \leq 5$), `verify`, `decompose`, `aggregate`, `branch`, `END`. Structural constraints: at most 8 nodes; at most 2 loops, each with `max_iter` ≤ 3; every node must reach `END` via non-loop edges, checked by reachability. Edge conditions come from a fixed grammar: `always`, `verify_passed`, `verify_failed`, `budget_below:`q, `budget_above:`q, with $q \in (0, 1)$ read against the ledger's most-binding remaining fraction. The meta-agent emits JSON patches only; no model-generated text is executed as program logic. Candidate solutions to code tasks are executed, but only inside a network-isolated subprocess with a 5 s CPU limit and a 4 GB address-space limit. Invalid candidates are rejected at zero API cost, and we report the invalid rate alongside search cost rather than hiding it.

B BUDGET SAFETY

Proposition 1. *Fix a task, a budget profile C, and any workflow expressible in the language. If (i) every metered call is preceded by a successful reservation of an upper bound on its billable vector, (ii) concurrent calls reserve their full aggregate before any of them starts, and (iii) every settlement reports actual usage no greater than its reservation, then cumulative usage never exceeds C in any countable dimension.*

Proof sketch. Induct on the number of settled calls, with invariant *used* + *reserved* $\leq C$ componentwise. It holds initially. A reservation is admitted only if the invariant survives adding its vector to *reserved*; a settlement moves a quantity no larger than the reservation from *reserved* to *used* and releases the remainder, which cannot increase the sum. Rejected reservations leave the state unchanged and divert control flow, so no unreserved call occurs. □

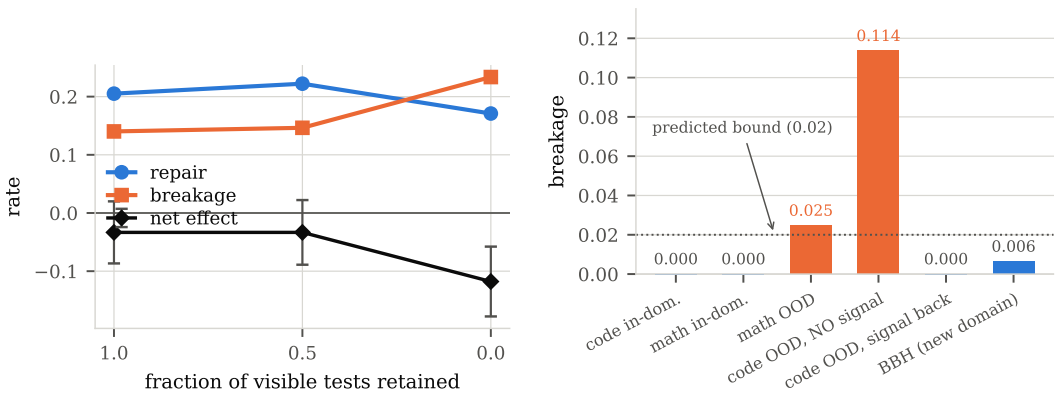

Figure 3: **Left:** withholding the verifier's signal is a causal manipulation of the breakage term. As visible tests are removed, breakage rises and repair falls, and the net effect drops to -0.118 $[-0.178, -0.058]$. **Right:** breakage of the incumbent-protecting construction across every condition we measured, including a third domain whose prediction was locked before execution. It stays at or near the predicted bound wherever the verifier carries signal and jumps to 0.114 in the single condition with none; restoring signal on the *same* tasks restores the guarantee.

Wall-clock time is not reservable, since its consumption is not known in advance; it is metered continuously and enforced by admission checks plus per-call deadlines. Provider-side cancellation latency is reported separately from policy-attributable overruns. A fuzz sweep of 3,000 randomly generated legal workflows against randomly drawn budget profiles, with a mocked model, produced zero cap violations in any dimension.

Assumption (iii) is an assumption about the *estimator*, and ours was initially wrong: a characters-per-token heuristic under-counted punctuation-dense code prompts by roughly 55%, so some settlements exceeded their own reservations. The implementation surfaces these as defects rather than absorbing them, which is how we found them. Appendix E quantifies the residual effect on the frozen test results, which were collected before the estimator was replaced with an exact tokeniser count plus a 5% margin.

C Failure taxonomy

Categories were fixed before inspection. Counts are over all failures in the frozen test split at seed 0, pooled across structures and budgets.

Category	share	example
first draft wrong, no refinement triggered	28.4%	`direct`, mbpp_103
premature budget exhaustion (reserve rejected)	26.3%	protected math, algebra_1071
verifier passed a wrong answer	19.8%	protected code, mbpp_113
ledger defect (settle overrun; see E)	18.4%	protected code, mbpp_578
refined but still wrong (signal insufficient)	7.1%	protected code, mbpp_160

The third row is the mechanism behind b: a verifier that accepts an incorrect answer cannot trigger the repair that would fix it, and one that rejects a correct answer invites the breakage that destroys it.

D Budget profiles and verifier signal

E Sensitivity to the ledger defect

4.7% of frozen-test executions $(466/9{,}900)$ terminated with a settlement overrun, unevenly across arms (protected 6.5%, vanilla 4.5%, baseline 3.6%), and were scored as failures. We recompute every headline contrast on the subset of tasks where *neither* arm hit the defect in *any* seed.

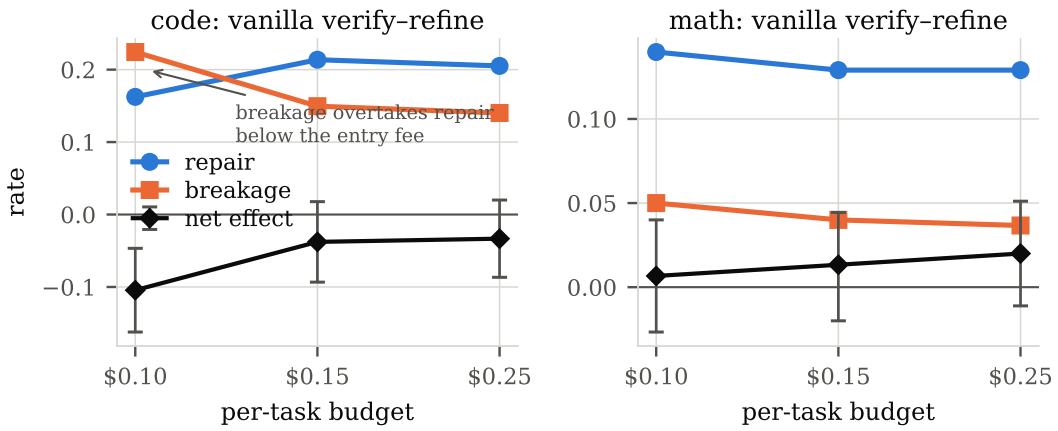

Figure 4: Both terms move together as the budget profile tightens: the workflow affords fewer refinement iterations, repair falls, breakage rises, and in the code family they cross where the net effect turns significantly negative. Profiles are named rather than dollar-valued because the dollar cap is not the binding constraint — realized spend is \$0.002–\$0.015 against caps of \$0.10–\$0.25, and admission failures are caused by the call and output-token caps. `tight` = (4 calls, 8k in, 2k out, \$0.10); `unseen` = (6, 12k, 3k, \$0.15); `loose` = (8, 16k, 4k, \$0.25). Which dimension binds is not identified by these composite profiles; see §8. Frozen test splits, 95% CIs on the net effect.

Contrast	all tasks	defect-free subset	n
C1 code loose, vanilla − direct	-0.033 $[-0.087, +0.020]$	-0.046 $[-0.096, +0.005]$	146
C2 code tight, vanilla − direct	-0.104 $[-0.162, -0.047]$	-0.115 $[-0.173, -0.056]$	148
C3 code loose, no signal	-0.118 $[-0.178, -0.058]$	-0.132 $[-0.192, -0.074]$	144
C4 code tight, protected	$+0.013$ $[+0.002, +0.031]$	$+0.014$ $[+0.002, +0.032]$	147
C4 code loose, protected	$+0.020$ $[+0.002, +0.042]$	$+0.018$ $[+0.002, +0.041]$	145
C4 code unseen, protected	$+0.020$ $[+0.002, +0.042]$	$+0.021$ $[+0.002, +0.043]$	146

Breakage of the protected construction is 0.000 under both views in every cell, so C4's structural claim is unaffected, and the C1–C3 harm effects become slightly *larger* on the defect-free subset, so those conclusions are not artifacts of the defect.

F MATH RE-GRADING

Our math grader compares a boxed answer by string match, then numerically, then by symbolic equivalence. The third stage never fired: the symbolic library was absent from the environment that produced the frozen runs, and the import sits inside an exception handler. Because every model response is stored, re-grading required no additional inference. 623 of 24,334 stored math answers (2.6%) changed grade, almost all of them formatting variants of the same value. The math baseline moved from 0.700 to 0.733. Two math sub-claims lost significance (protected at `unseen` and at `loose`); no verdict changed, and breakage remained 0.000 in every in-domain protected cell. All numbers in this paper use the corrected grading; the original field is preserved on disk.

G AUTOMATED SEARCH: PROTOCOL AND PER-CLAIM RESULTS

Incumbent protection was derived by us, from our own decomposition, and tested by us. We therefore asked whether a search procedure, given a design space we specified in advance and no access to that derivation, selects the same construction. It is a bound on our design bias, not an independent replication: the node whitelist, the edge grammar and the objective are ours.

The searcher evolves workflows in the restricted language of Appendix A under multi-fidelity racing (Jamieson & Talwalkar, 2016), scoring candidates jointly at both searched budgets. Its budget-contingent and static arms differ only in whether edges may read remaining budget. At full fidelity

on the search split every arm selected the same shape—a low-temperature draft, a verifier gate, refinement only on rejection. We no longer read this as the search having rediscovered our principle. Every candidate in the space carries the verifier gate, so selecting it is not evidence about the mechanism of equation 4; and the space does not contain the choice that matters, namely whether to reuse a reference system's stored output as the incumbent. What the search does show is a preference for low-temperature drafts, which is consistent with, but does not isolate, reference preservation. Of five preregistered confirmatory claims (`prereg-freeze-partII`), three fail: non-inferiority holds in code at both budgets and at math/`loose` but not at math/`tight`; the cost reduction reaches 10.5% against a 20% threshold, and in math the searched policy is 191% *more* expensive in exchange for $+0.049$ accuracy, a Pareto trade rather than a saving; and only 3 of 12 selected policies retain the gated shape, because the preregistered one-shot selection uses a 40-task validation split whose variance swamps the 1–2 point gaps between candidates. What survives is narrow: at an unseen \$0.15 budget the code policy is indistinguishable from the preregistered static control (-0.001 $[-0.013, +0.010]$) at half the search cost, and a search over our design space does select the gated pattern there. Appendix G gives the full protocol and per-claim results.

Incumbent protection was derived by us, from our own decomposition, and tested by us. We therefore built a searcher to check whether a procedure with no access to that reasoning selects the same construction from a space we specified in advance.

Candidates are workflows in a restricted language: at most eight nodes, at most two loops of at most three iterations, a fixed node whitelist, no model-generated executable code, and static validation before any spend (Appendix A). Search is evolutionary with multi-fidelity racing over nested task subsets (Jamieson & Talwalkar, 2016), evaluating every candidate jointly at both searched budgets and ranking by a score that averages per-budget accuracy with the worst budget, so a candidate cannot win by specialising. The budget-contingent arm and the static control differ only in whether edges may condition on remaining budget; over 200 sampled workflows the static arm never emitted a budget predicate. Protocol A gives the static control a full search budget *per* budget tier, twice what the budget-contingent arm receives.

Selection variance in Part II. Our preregistered rule selects a policy once on a 40-task validation split, which cannot resolve the 1–2 point gaps between candidates—hence shape convergence is universal in the search-split analysis but $3/12$ among selected policies. A larger validation split is the fix; we did not retrofit it after the freeze.

Restricted design space, coarse verifier taxonomy. Results about "structure" are results about a space of at most eight nodes and two loops with a fixed whitelist, and we treat verifiers as oracle, non-oracle-with-signal or no-signal, whereas real verifiers vary continuously in sensitivity and specificity. equation 2 folds both into r and b; separating them would give a sharper rule and is the obvious next step.

H SEARCH DETAILS

A note on the racing rule. Our first implementation compared a candidate's lower confidence bound against the incumbent's with a fixed slack of 0.10. The Hoeffding half-width (Hoeffding, 1963) differs by 0.107 between the $n{=}24$ and $n{=}64$ rungs, so promotion from the cheapest rung was arithmetically impossible and almost no candidate ever left it. Whenever rung sizes differ, racing must compare a candidate's *upper* bound against the incumbent's lower bound; a fixed slack cannot substitute for the difference in interval widths. No result reported in this paper comes from runs made before this was corrected.

Fidelity rungs are nested prefixes of the search split: 24, 64 and 120 tasks, with 1, 1 and 2 execution seeds. Racing eliminates a candidate only when its Hoeffding upper bound falls below the incumbent's lower bound at the same or a higher rung. Selection and archive ranking use a fidelity-comparable mean-based score; the conservative bound is used only for the racing rule.

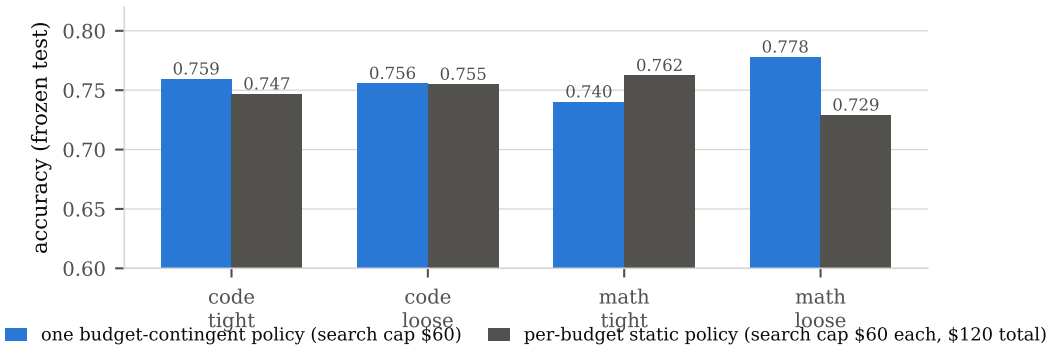

Figure 5: Protocol A on the frozen test split: one budget-contingent policy against the static policy searched specifically for each budget, which received twice the total search budget. We deliberately do not plot the two search procedures' internal scores against each other—the budget-contingent score is a cross-budget minimum and the static score is a single-budget value, so one axis would compare incomparable quantities. Deployment accuracy at a given budget is comparable, and is what is shown. Code figures average three search seeds.

Run (code, search seed 0)	candidates	spend	rung distribution	archive
budget-contingent	33	$63.0	9/5/19	11
static @ tight	76	$60.7	35/5/36	38
static @ loose	30	$61.6	5/5/20	8

I Limitations bearing on scope

Verification differs in kind across our families, and our taxonomy of verifiers is coarse. Code has unit tests; math and logic have a gold-free self-check, a different object. We treat verifiers as oracle, signal-bearing or signal-free, whereas real ones vary continuously in sensitivity and specificity, and equation 2 folds both into r and b. Results about "structure" are results about our restricted language (Appendix K).

The budget floor is not identified to a resource. Our profiles move call, token and dollar caps together, and the dollar cap is not the binding one: realized spend is \$0.002–\$0.015 against caps of \$0.10–\$0.25, and admission failures come from the call and output-token caps. A single-dimension sweep would be needed to name the resource; we report a crossover across composite profiles instead.

Estimating the bound is not free. Obtaining $\Pr(R \mid B \text{ correct})$ needs the reference's outputs, labels or a grader identifying the correct ones, and data representative of deployment. When the baseline is strong the conditioning set is small — 107 tasks at code/`loose` — so a useful upper bound needs more labelled data than "measure your verifier" suggests. We report one-sided Clopper–Pearson upper bounds and recommend those, not point estimates, for planning. At our sample sizes they are wide: no condition we ran certifies a false-rejection rate below 0.03, so the gate we propose licenses refinement only for deployments that tolerate percent-level breakage risk. A resampling bound is not an alternative — with zero observed events every resample is zero too, so a percentile bootstrap returns exactly 0, which is not a bound.

J Measurement and engineering reliability

Two defects left traces in the released data. A token estimator that under-counted punctuation-dense prompts caused 4.7% of frozen-test executions to be scored as failures, unevenly across arms, and our math grader's symbolic-equivalence branch never executed, misgrading 2.6% of math answers. Both are fixed. Re-grading every stored response at no inference cost moved the math baseline from 0.700 to 0.733 and cost two math sub-claims their significance; recomputing every

Table 4: Breakage of the reference-preserving workflow across verifier regimes and domains. Consistent with equation 5, what varies is the verifier rather than the domain: the one condition whose verifier has no signal is an order of magnitude worse than the rest, and restoring signal on the same tasks restores the result.

Condition	Verifier	breakage	Δ (95% CI)
in-domain code (MBPP+)	oracle tests	0.000	$+0.020$ $[+0.002, +0.042]$
in-domain math (MATH)	non-oracle check	0.000	$+0.007$ $[-0.002, +0.018]$
out-of-domain math (held-out subjects)	non-oracle check	0.025	-0.003 $[-0.027, +0.017]$
out-of-domain code (HumanEval+)	*none*	0.114	-0.087 $[-0.140, -0.033]$
out-of-domain code, signal restored	recovered tests	0.000	$+0.005$ $[+0.000, +0.015]$
new domain (BIG-Bench Hard)	non-oracle check	0.006	$+0.025$ $[+0.003, +0.050]$

headline contrast on the defect-free subset leaves the breakage results unchanged and the harm effects slightly larger (Appendices E and F). Both gradings are in the released data. Appendix L audits what response caching did and did not affect.

Selection variance in the search study. A one-shot selection on a 40-task validation split cannot resolve the 1–2 point gaps between candidates, which is why the shape that dominates the search-split analysis appears in only 3 of 12 selected policies. We did not retrofit a larger split after the freeze. We also cannot explain with data why the searched policy is better at math/`loose` and worse at math/`tight`.

K Scope of the design space and verifier taxonomy

No mechanism for the math search boundary. The budget-contingent policy is better at math/`loose` and worse at math/`tight`, and we cannot explain this with data. It would be easy to present it as a boundary predicted by equation 3; the measured ratio sits essentially on the threshold, so the rule is silent there and does not predict harm. We do not claim it does.

L What response caching did and did not affect

Caching was enabled for the structure-library runs (Appendix N), which is how the reference-preserving arm came to hold a byte-identical copy of the baseline's output. A reader is entitled to ask what else it touched. We audited three questions against the implementation and the stored logs.

Were the three execution seeds collapsed into one? No. The cache key is a hash over the deployment name, the message list, and a parameter dictionary that includes the execution seed, so runs at different seeds cannot share an entry. Empirically, across the frozen test split, 102–150 of 150 tasks per cell produce different outputs across the three seeds. The seed-averaging and the intervals built on it therefore rest on genuinely repeated executions.

Did a cache hit escape the budget ledger? No. The reservation is taken *before* the cache is consulted, and a hit is settled against that reservation with the stored token counts, so a cached call consumes call count, tokens, and dollars exactly as a fresh one does and can be refused by the same admission check. The budget-floor result of §5 is consequently not an artifact of caching; what caching changes is the invoice and the wall clock, neither of which carries a claim in this paper.

What caching did change. Two things. It made the invoice roughly a quarter of the logical figure. And, because the baseline's call and the reference-preserving arm's first call are configured identically at a given seed, it made them resolve to one entry — which is why $\Pr[I{=}B] = 1$ holds exactly rather than approximately. That is a coupling between arms, and we treat it as part of the treatment definition rather than as an incidental detail: the comparison is between an arm holding the reference's own output and an arm holding a different draft, and caching is the mechanism that

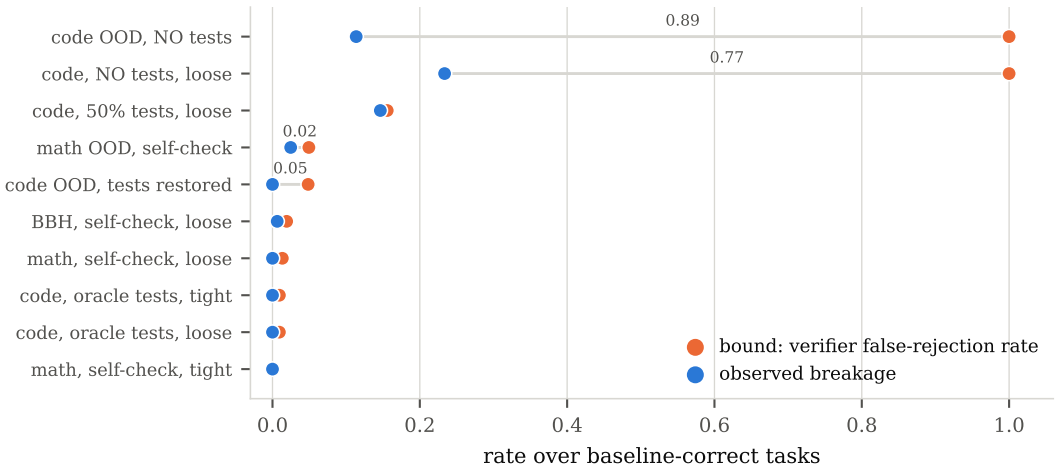

Figure 6: The bound equation 5 against measurement, in every condition we ran. Its left-hand side is a property of the verifier alone and can be measured without executing the workflow: a gated workflow that exits after its verifier accepted the draft leaves a different trace from one that proceeds to refine, so the false-rejection rate is recoverable from logs already collected. Observed breakage falls below the bound in all ten conditions — as it must, if the premise holds — across the full range from a verifier that almost never rejects a correct draft (0.009) to one that rejects every draft because it has no information (1.000). The figure is a check on the implementation and a reading of the bound's slack, not a test of a contingent law. The bound is nearly tight where the verifier is moderately noisy—breakage 0.146 against a bound of 0.156 under 50% test masking—and loose where it is uninformative, because refinement sometimes recovers an answer it has just discarded. Rates are over baseline-correct tasks.

made the first arm's incumbent exact. A design that wants the property rather than inheriting it should assign the stored reference output directly, as §8 notes.

What regeneration under the same policy would have cost (recovered from stored seeds, no new inference). §8 asks for a three-arm provenance comparison we did not run. One of the missing arms — regeneration under the same policy as opposed to reuse of the stored reference output — is already present in the data: the cache key includes the execution seed, so the identical prompt at temperature 0 was independently regenerated three times per task. Table 5 treats every ordered pair of seeds on a task as one reference draft and one regeneration. `identical` is how often the two seeds' text matches byte for byte; `leak` is $\Pr(\text{regenerated draft wrong} \mid \text{reference draft correct})$, task-clustered with a bootstrap CI; `accept|I wrong` is the measured acceptance rate on tasks where the baseline fails on every seed; and `leak×accept` estimates the accepting-path term $\Pr(\text{accept}, I \text{ wrong} \mid B \text{ correct})$ in equation 4 that a regenerating arm would carry, against the 0 reference preservation delivers by construction.

Table 5: Regeneration leak, by domain, computed from the three stored execution seeds with no new inference.

Domain	tasks	identical	leak	[95% CI]	accept|I wrong
code	111	0.447	0.027	[0.005, 0.059]	0.613
math	119	0.111	0.113	[0.067, 0.164]	0.417
BBH	115	0.061	0.061	[0.026, 0.100]	0.600
code OOD	90	0.560	0.011	[0.000, 0.028]	0.000
math OOD	47	0.000	0.266	[0.170, 0.362]	0.138

Two readings. First, temperature 0 is far from deterministic in this deployment: identical text ranges from 0% (math OOD) to 56% (code OOD) across seeds, so "configure the regenerator like the baseline" does not by itself deliver $I = B$, consistent with the caution in §5. Second, the accepting-path exposure this would create is domain-dependent and non-trivial: roughly 1.7% in code and

4.7% in math, against the 0% observed under reference preservation. This is not the causal test §8 still calls for — both seeds here share the exact prompt and temperature of the baseline, so it cannot distinguish a regenerating policy from a differently-drafting one, which is the third arm we still lack — but it does rule out the objection that reference preservation's zero is a nominal artifact of an already-near-deterministic generator.

What regeneration under a genuinely different policy costs (one real generation call per task, no verify or refine loop). The remaining arm needed new data: a single `generate` node under the vanilla workflow's actual first-draft configuration (`solve_cot`, temperature 0.7, 1536-token cap), on the same frozen test tasks, three seeds, no verify or refine node attached, so the raw draft is exactly what would have entered the vanilla arm's verifier. Table 6 reports the same quantities as Table 5 for code and math. `accept|I wrong` for code was obtained at zero additional cost: code's verifier is the oracle feedback-test suite, a subprocess execution rather than a call, so it costs nothing to ask what it would have decided on this arm's wrong drafts. Math's verifier is an LLM self-check and is not reported here; obtaining it needs new inference beyond what this arm's generation already used.

Table 6: Accepting-path exposure under a different drafting policy (`solve_cot`, temperature 0.7), on the same baseline-correct tasks Table 1 conditions on.

Domain	baseline-correct n	leak	[95% CI]	accept|I wrong
code	107	0.224	[0.168, 0.284]	0.490
math	100	0.053	[0.030, 0.083]	—

A genuinely different policy carries far more accepting-path exposure than same-policy noise in code: leak is roughly eight times higher than Table 5's regeneration figure (0.224 against 0.027), and the resulting exposure estimate, leak times `accept|I wrong`, is 0.110 against 0.017 — both against the 0.000 reference preservation delivers by holding $I = B$ instead of regenerating under either policy. Math moves the other way: leak is lower under the different policy than under same-policy regeneration (0.053 against 0.113), so for math this arm alone does not support a "more different, more exposure" reading, and without `accept|I wrong` the domain's accepting-path exposure cannot be estimated from this table at all. Read together with Appendix L's same-policy result, the two zero-and-low-cost arms agree that provenance carries real, non-nominal exposure in code and disagree on its direction relative to same-policy noise in math — which is itself informative about how domain-specific this mechanism is, and is consistent with what the three-arm causal test in §4 finds directly: explicit reuse closes the accepting path in both domains, but the two regeneration policies order oppositely by domain.

M A SECOND MODEL FAMILY

Everything above is one model. As a check on whether the qualitative pattern — breakage near zero under a reliable verifier, rising as verifier signal degrades — is specific to GPT-4o, we replicated all ten of Table 1's conditions on Kimi K3, a different model family. An initial pass had uneven API call failures (0–56% per cell); a frozen backfill protocol (retry only the calls that returned a provider error, same model/prompt/temperature/caps/seed, concurrency 2, fixed backoff, logged append-only, never touching an already-completed call) brought every condition to 97–100% completion, logged in PREREGISTRATION.md §9. We report all ten, with completion rate, rather than filtering by a reliability threshold.

The qualitative pattern replicates, cell for cell: breakage is at or near zero under oracle tests and under tight self-check (0.000, 0.000, 0.008, 0.000), rises as visible tests are masked (0.000 $\rightarrow$ 0.046 $\rightarrow$ 0.152), peaks under the two no-signal conditions (0.104, 0.152), and falls back to 0.010 when signal is restored on the same OOD tasks — the same ordering GPT-4o shows in Table 1, though at different magnitudes (e.g. code/no-tests: 0.152 here against 0.234 there). We read this as a second-family replication of the qualitative verifier-sensitivity pattern, not of the causal test, which we did not rerun on a second model, and not of the magnitudes, which our transfer tests (§7) already show are not portable even within GPT-4o.

Table 7: Kimi K3, all ten of Table 1's conditions. Completion is the protected arm's fraction of task×seed calls that returned a model response (baseline completion was $\geq 99.6\%$ throughout).

Condition	p(baseline)	repair	breakage	net	completion
code, oracle, loose	0.771	0.317	0.000	+0.047	100%
code, oracle, tight	0.773	0.150	0.000	+0.000	99%
math, self-check, loose	0.807	0.252	0.008	+0.002	100%
math, self-check, tight	0.804	0.216	0.000	+0.002	98%
BBH, self-check, loose	0.950	0.095	0.014	−0.017	100%
code OOD, no tests	0.867	0.383	0.104	−0.087	100%
math OOD, self-check	0.440	0.186	0.030	−0.010	100%
code OOD, tests restored	0.926	0.667	0.010	+0.025	100%
code, 50% tests, loose	0.773	0.358	0.046	+0.018	100%
code, no tests, loose	0.773	0.225	0.152	−0.103	97%

N COST ACCOUNTING

We separate two cost figures that are easy to conflate.

Per-task deployment cost — every dollar figure in the tables — is a *logical* cost: what the workflow would cost with no response caching. This is the quantity a deployment faces, because a deployed system meets new tasks and cannot serve them from a cache of its own past answers. Reporting cache-assisted prices in those tables would understate deployment cost substantially.

Study cost is the actual invoice, which is much smaller. Response caching was enabled for the structure-library runs, including those on the frozen test split, so identical calls across arms resolved to a single API request. The ledger charges a cached call exactly as it charges a fresh one, which is why no reported quantity changes; but it also means the two figures diverge by the cache hit rate. Summed over every experiment reported here the logical figure is \$1,317.74, against \$2,000 as the ledger's circuit-breaker cap, while the billed figure is roughly \$300 — consistent with 77,942 unique cached responses serving 194,318 task executions. Only the searched-policy evaluations of Appendix G ran with caching disabled.

Caching also has a methodological consequence used elsewhere in this paper: it is why identically configured calls returned identical text, and therefore why $\Pr[I{=}B] = 1$ held exactly rather than approximately (§4).

O PREREGISTRATION AND OUTCOMES

The repository contains the preregistration with an append-only deviation log. Claims were committed and git-tagged before the corresponding frozen split was executed. Outcomes as recorded: Part I C1–C3 supported, C4 unsupported as preregistered (the comparison did not manipulate the registered variable) and C5 refuted; Part II P1–P3 not supported as stated, P4 weaker than on the search split, P5 reported; the locked out-of-domain transfer prediction failed; the prospective predictions PV1 and PV3 held and PV2 did not.